Università di Pisa

Dipartimento di Informatica
Corso di Laurea Magistrale in Informatica

Compressed Representations of Set Collections

Candidato
Luca Lombardo

Relatore
Roberto Grossi

Anno Accademico 2025/2026

Contents

Contents ii

1 Introduction **1**
1.1 Why compress set collections? 1
1.2 Set collections 4
1.3 Contribution and roadmap 6

Bibliography **7**

List of Figures

Introduction 1

1.1 Why compress set collections? 1
1.2 Set collections 4
1.3 Contribution and roadmap 6

1.1 Why compress set collections?

Every finite object has a shortest description. Given a universal Turing machine $\mathcal{U}$, the *Kolmogorov complexity* $K(x)$ of a binary string x is the length of the shortest program p such that $\mathcal{U}(p) = x$[1]. This quantity captures the intrinsic information content of x. A string whose shortest description is nearly as long as the string itself is incompressible, while one admitting a substantially shorter program is compressible. Kolmogorov complexity does not depend on the choice of compression scheme, only on the choice of universal machine, and even that dependence vanishes up to an additive constant[1].

[1]: Li et al. (2008), *An introduction to Kolmogorov complexity and its applications*

1: The *invariance theorem* states that for any two universal machines $\mathcal{U}_1$ and $\mathcal{U}_2$, the corresponding complexities satisfy $|K_{\mathcal{U}_1}(x) - K_{\mathcal{U}_2}(x)| \leq c$ for a constant c independent of x.

Kolmogorov complexity is, however, not computable[1]. No algorithm can determine, given an arbitrary string, whether a shorter program exists. We cannot even approximate $K(x)$ from above in general, since doing so would require solving the halting problem. Any practical compression scheme must identify specific structural regularities in the data (patterns, redundancies, correlations) and design representations that exploit them.

[1]: Li et al. (2008), *An introduction to Kolmogorov complexity and its applications*

This observation shapes the way we think about data compression in the context of data structures. A classical compression algorithm treats data as a passive archive, compressing it as tightly as possible for storage and decompressing it in full before use. A *compact data structure*, by contrast, maintains the data in a compressed form that supports direct queries (membership tests, rank computations, element retrieval) without decompression[2]. The representation must use space close to a well-defined lower bound and support the desired operations efficiently. The formal study of compact data structures can be traced to the work of Jacobson[3], who showed that trees and graphs admit representations in space matching information-theoretic lower bounds while supporting navigational queries efficiently. Since then, the field has produced compact representations for bitvectors, sequences, permutations, trees, graphs, and text indexes, each measured against an appropriate entropy baseline.

The tension between the incomputability of $K(x)$ and the practical need for compression is the starting point of the entire field of compact data structures. We do not aim for the shortest description in the absolute sense; we aim for representations that are short *relative to a well-defined compressibility measure* while supporting efficient queries.

[2]: Navarro (2016), *Compact data structures: A practical approach*

[3]: Jacobson (1989), *Space-efficient static trees and graphs*

What constitutes the *right* lower bound depends on the data and the queries we wish to support. For a single set S of n elements drawn from a universe U of size u, the number of possible such sets is $\binom{u}{n}$, and any encoding must use at least $\lg \binom{u}{n}$ bits to distinguish all possibilities.[2] This quantity is both the worst-case lower bound and the independent encoding cost for a single set. But when we have not one set but a *collection* of sets, the situation changes. Inter-set relationships may make the collection far more compressible than the sum of its parts. Identifying the right compressibility measure for a collection, and building data structures that achieve it, is the central problem of this thesis.

2: Throughout this thesis, lg denotes $\log_2$.

Data takes the form of a collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ of finite subsets of a shared universe U in many application domains. Three examples illustrate the scale of the problem and the correlations that independent encoding ignores.

Web graphs

Boldi and Vigna's WebGraph framework remains the standard for billion-scale graph compression, more than twenty years after its introduction.

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

A *web graph* is a directed graph in which each node represents a URL and each arc represents a hyperlink[4]. The adjacency list of a node (the set of its successors) is a subset of the vertex set, and the entire graph is naturally viewed as a collection of sets: one per node, drawn from the shared universe of all URLs. At the time of Boldi and Vigna's 2004 paper, the World Wide Web contained an estimated three billion nodes and more than fifty billion arcs, and these figures were only lower bounds, since they came from search engines that index only a fraction of the Web.

The naive representation of a graph on N nodes stores each adjacency list as a sorted sequence of node identifiers, preceded by the outdegree. For the WebBase graph, a crawl of 118 million pages and one billion links, this representation is prohibitive. Boldi and Vigna identified two key empirical regularities. *Locality* means that most links are navigational, connecting pages within the same site, so that source and target URLs share long common prefixes and, when sorted lexicographically, have indices close to each other. *Similarity* means that pages close in lexicographic order share large segments of their successor lists, because many navigational links point to the same fixed set of pages within a site hierarchy.

Similarity in web graphs means that the *symmetric difference* between neighbouring adjacency lists is small. This is precisely the regularity that symmetric-difference compression exploits.

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

The WebGraph framework exploits these regularities through *reference compression*: the adjacency list $S(x)$ of node x is encoded relative to the list $S(y)$ of a nearby reference node y, via a *copy list* indicating which elements of $S(y)$ also appear in $S(x)$, together with the remaining elements encoded using gap compression. On graphs with hundreds of millions of nodes and billions of links, WebGraph achieved compression ratios of a few bits per link[4]. The key observation is that adjacent adjacency lists, viewed as sets of node identifiers, are highly correlated, and their symmetric differences are small relative to the individual sets.

Colored de Bruijn graphs in pangenomics

A colored de Bruijn graph indexes multiple genomes simultaneously. The *color set* of a k-mer records which reference genomes contain it.

[5]: Alanko et al. (2023), *Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes*

A *de Bruijn graph* of order k for a collection of strings is a directed graph whose vertices are the distinct k-mers (substrings of length k) and whose edges connect consecutive k-mers in the input[5]. In computational pangenomics, a *colored* de Bruijn graph augments this structure with color information: each k-mer is associated with a *color set*, the set of identifiers of the reference genomes that contain it. The color sets are subsets of a shared universe $\{1, \ldots, m\}$, where m is the number of reference genomes, and the collection of all color sets is the data structure that must be stored compactly and queried efficiently.

[5]: Alanko et al. (2023), *Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes*

The scale of modern pangenomic datasets makes compact representation essential. Alanko et al. evaluated Themisto, a colored k-mer index, on collections of up to hundreds of thousands of *Salmonella enterica* genomes[5]. At these scales, competing tools either failed to complete within practical time limits or required an order of magnitude more memory.

The color sets exhibit strong inter-set correlation. The most common color-set cardinality is 2, and the distribution of color-set sizes follows a power law. Most color sets are small, and a few very large color sets dominate the total space. Neighbouring k-mers in the de Bruijn graph tend to have identical or nearly identical color sets, because a mutation in a single genome affects only a few consecutive k-mers while leaving the

rest of the color information unchanged. Almodaresi et al.[6] exploited this graph-topological locality by computing a minimum spanning tree over a sparse similarity graph induced by the de Bruijn graph structure, encoding each color set as a symmetric difference with its neighbour on the tree.[3]

[6]: Almodaresi et al. (2020), *An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search*

3: The containment entropy framework of Alanko et al. [7] was later tested on Themisto's color sets, achieving 0.18 bits per element compared to Themisto's default of 0.32 bits per element.

Each color set is a subset of $\{1, \ldots, m\}$, where the universe is the set of genome identifiers. The collection of all color sets (one per k-mer) must support membership queries (given a genome i and a k-mer x, decide whether x appears in genome i) and retrieval (list all genomes containing a given k-mer) in compressed space.

Inverted indexes in information retrieval

An *inverted index* maps each term in a text collection to its *posting list*: the set of documents in which the term appears. If the collection contains u documents and the vocabulary has m terms, the inverted index is a collection of m sets drawn from the universe $\{1, \ldots, u\}$. Bookstein and Klein[8] observed that posting lists consume large amounts of storage and that inter-list correlations offer an underexploited avenue for compression.

The insight that posting lists for semantically related words exhibit similar membership patterns dates to the earliest days of information retrieval.

[8]: Bookstein et al. (1991), *Compression of correlated bit-vectors*

Certain words are strongly correlated across documents. If one word appears in a particular document, semantically related words are likely to appear in the same document. Bookstein and Klein proposed to exploit this correlation by compressing collections of bitvectors (the characteristic vectors of the posting lists) using their symmetric differences. If two bitvectors X_1 and X_2 are *close* in Hamming distance, then $X_1 \oplus X_2$ is a sparse bitvector that compresses well. The original X_2 can be recovered from X_1 and $X_1 \oplus X_2$.

The problem has a clean graph-theoretic structure. The bitvectors form the vertices of a complete graph, with edge weights given by Hamming distances. Introducing a zero vector X_0 (corresponding to the empty set) and computing a minimum spanning tree on this augmented graph yields the optimal clustering. Each root bitvector is stored directly, and each non-root bitvector is stored as the symmetric difference with its parent on the tree.

Bookstein and Klein showed that this problem reduces to computing a minimum spanning tree on $m + 1$ vertices, solvable in $O(m^2 \log m)$ time. Their experiments on the Hebrew Bible showed that clustering nearly doubled the compression savings for some sets of bitmaps.

This 1991 paper is the earliest treatment, in the literature on set collections, of the principle that underlies all subsequent work on symmetric-difference compression. The idea was rediscovered independently by Adler and Mitzenmacher for web graph compression, by Bharat et al. for web link information, by Buchsbaum et al. for massive tabular data, and by Björklund and Lingas for Boolean matrix multiplication[9]. In each case, the application-specific implementations restricted the graph structure for efficiency (Boldi and Vigna[4] to a window of W consecutive nodes, Almodaresi et al. to edges between neighbouring k-mers in the de Bruijn graph), sacrificing global optimality for practical speed.

Despite more than thirty years of applied work on symmetric-difference compression, a formal compressibility measure for set collections, with provable bounds on the achievable space and query time, was proposed only recently.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

The question this thesis addresses

All three domains represent data as a collection of sets over a shared universe. Encoding each set independently costs $H_{wc}(\mathcal{S}) = \sum_i \lg \binom{u}{|S_i|}$ bits, ignoring the correlations between sets that the examples above make evident. What compressibility measure should we use for a collection of sets, and which queries can we support on the compressed representation?

The worst-case entropy $H_{wc}(\mathcal{S})$ treats each set in isolation. The inter-set correlations visible in all three domains suggest that the collection is far more compressible than this baseline.

4: Membership, access, rank, predecessor, and successor are the fundamental primitives for higher-level tasks such as set intersection and union.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

The [4] answer depends on which structural regularities we choose to exploit. If some sets are subsets of others, we can encode a smaller set cheaply relative to a larger one. Alanko, Bille, Gørtz, Navarro, and Puglisi[7] call the resulting measure the *containment entropy*. If sets that are neither subsets nor supersets of each other nevertheless have small symmetric differences, we can encode one in terms of the other via insertions and deletions. Gagie, He, and Navarro[9] define the *symdiff compressibility* to capture this. Both measures improve upon $H_{wc}(\mathcal{S})$ under the right conditions, and both yield compressed representations supporting membership, access, rank, predecessor, and successor queries with logarithmic time bounds. Both also have limitations that leave room for further improvement.

The contribution of this thesis is a polynomial-time algorithm, *Set-Union Matching*, that builds compressed hierarchies by introducing synthetic union nodes. The encoding cost is guaranteed to be at most $H_{wc}(\mathcal{S})$.

This thesis proposes a third compressibility measure, based on augmenting the collection with synthetic sets (unions of pairs) that serve as common ancestors in a hierarchy. The resulting measure, $L_{\mathrm{SUM}}(\mathcal{S})$, is never worse than $H_{wc}(\mathcal{S})$, and the hierarchy supports the same five operations with logarithmic query times.

1.2 Set collections

We move from the concrete applications of Section 1.1 to the abstract structure they share. The definitions in this section apply uniformly to web graphs, colored de Bruijn graphs, and inverted indexes.

The three domains in Section 1.1 share a common abstraction. Data takes the form of many sets over a shared universe, and the question of how to represent them compactly reduces to understanding what it means for a collection of sets to be compressible. We now state the problem precisely.

Definition 1.2.1 (Set collection) *A set collection is a tuple $(\mathcal{S}, U)$, where U is a finite, totally ordered set called the universe, and $\mathcal{S} = \{S_1, \ldots, S_m\}$ is a family of m finite subsets of U. We write $u = |U|$ for the size of the universe and $n = \sum_{i=1}^{m} |S_i|$ for the total size of the collection. We assume $U = [1..u] = \{1, 2, \ldots, u\}$ without loss of generality, since any totally ordered set of size u can be mapped to this interval by a rank function computable in $O(1)$ time after $O(n \lg u)$-time preprocessing*[5].

5: If the universe is not $[1..u]$, one collects the n elements, sorts them, and assigns integers in $[1..u]$ to the u distinct values. Queries and answers translate in $O(1)$ time via a lookup table.

The total order on U is what makes rank, predecessor, and successor well-defined. Without it, only membership and access would make sense.

Each set S_i inherits the total order from U, so its elements can be listed as $S_i[1] < S_i[2] < \cdots < S_i[|S_i|]$. The integer n counts elements with multiplicity across sets; an element belonging to k sets contributes k to the sum. In practice $n \ll m \cdot u$, because most sets are small relative to the universe. In the web graph example from Section 1.1, a node with ten outgoing links contributes a set of size ten to the collection, even though the universe contains hundreds of millions of URLs.

Gagie, He, and Navarro [9] state the same five operations in their Section 1. Alanko et al. [7] work with the same operations in a bitvector formulation that is equivalent after identifying each set with its characteristic vector.

A representation of $\mathcal{S}$ must support queries that allow us to recover information about any individual set S_i without decompressing the entire collection.

Definition 1.2.2 (Operations on set collections) *Given a set collection $(\mathcal{S}, U)$ with $U = [1..u]$, we define five operations on each $S \in \mathcal{S}$:*

- $\mathrm{member}(S, x)$*: returns true if $x \in S$ and false otherwise.*
- $\mathrm{access}(S, i)$*: given $i \in [1..|S|]$, returns the i-th smallest element of S, that is, $S[i]$.*
- $\mathrm{rank}(S, x)$*: returns $|\{y \in S : y \leq x\}|$, the number of elements of S at most x.*
- $\mathrm{predecessor}(S, x)$*: returns $\max\{y \in S : y \leq x\}$, or $-\infty$ if no such element exists.*

- successor(S, x): *returns* $\min\{y \in S : y \geq x\}$, *or* $+\infty$ *if no such element exists.*

These five operations are the primitives on which higher-level tasks depend. Set intersection, for instance, can be computed by interleaving predecessor and successor queries on both operands, advancing through the sorted elements in tandem.

Predecessor [6] and successor reduce to rank followed by access. A predecessor query performs one rank computation and one access computation; a successor query does the same. The three operations member, rank, and access are therefore the independent primitives. Any representation that supports these three automatically supports all five, at the cost of one additional access per predecessor or successor query.

6: To compute predecessor(S, x), first compute $r = \text{rank}(S, x)$. If $r = 0$, no predecessor exists. Otherwise, return access(S, r). Successor works symmetrically.

We can now ask how many bits are needed to store the collection $\mathcal{S}$. A natural starting point treats each set independently. A set $S_i \subseteq U$ of size $|S_i|$ is one of $\binom{u}{|S_i|}$ possible subsets of U of that size. Any encoding that can distinguish all such subsets requires at least $\lg \binom{u}{|S_i|}$ bits for S_i.

The binomial counting argument is elementary but central. It establishes the baseline against which every compressed representation in this thesis is measured.

Definition 1.2.3 (Independent encoding cost) *The independent encoding cost (or worst-case entropy) of a set collection* $(\mathcal{S}, U)$ *is*

$$H_{wc}(\mathcal{S}) \;=\; \sum_{i=1}^{m} \lg \binom{u}{|S_i|}.$$

Alanko, Bille, Gørtz, Navarro, and Puglisi call this quantity the worst-case entropy[7], writing it as $H(\mathcal{S})$; we adopt $H_{wc}(\mathcal{S})$ to distinguish it from the finer compressibility measures introduced later. The name reflects that $\lg \binom{u}{|S_i|}$ is the number of bits needed to encode a subset of size $|S_i|$ from a universe of size u, without any assumptions about the structure of the specific set. We work in the word RAM model with word size $\omega \geq \lg u$, so each universe element fits in a single machine word, and space is measured in bits throughout. The bound $H_{wc}(\mathcal{S})$ is achievable: encoding each set independently costs $H_{wc}(\mathcal{S}) + O(n + m \lg n)$ bits in total[7]

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

7: The $O(m \lg n)$ term accounts for index pointers; the $O(n)$ term for per-set overhead.

The three applications in Section 1.1 show, however, that the sets in a collection are rarely unrelated. Adjacent adjacency lists in a web graph share most of their links. Neighbouring k-mers in a de Bruijn graph have nearly identical color sets. Posting lists for semantically related terms overlap heavily. In each case, $H_{wc}(\mathcal{S})$ treats each set as if it were drawn independently from $\binom{u}{|S_i|}$ possibilities, ignoring the correlations that make the collection far more compressible than the sum of its parts. If $S_i \subset S_j$, we can describe S_i as a subset of S_j in $\lg \binom{|S_j|}{|S_i|}$ bits, which is much less than $\lg \binom{u}{|S_i|}$ when $|S_j| \ll u$[7]. If S_i and S_j have small symmetric difference, we can describe one in terms of the other cheaply[9]. Either way, the encoding cost drops below $H_{wc}(\mathcal{S})$.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Two questions arise naturally. The first is combinatorial. Which compressibility measure captures the inter-set relationships of a given collection, and how much smaller can the encoding be compared to $H_{wc}(\mathcal{S})$? The second is algorithmic. Can we build data structures that achieve the compressed space and still support member, access, rank, predecessor, and successor queries efficiently? Section 1.3 outlines our answers.

1.3 Contribution and roadmap

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Bibliography

[1] Ming Li, Paul Vitányi, et al. *An introduction to Kolmogorov complexity and its applications*. Vol. 3. Springer, 2008 (cited on page 1).

[2] Gonzalo Navarro. *Compact data structures: A practical approach*. Cambridge University Press, 2016 (cited on page 1).

[3] Guy Jacobson. 'Space-efficient static trees and graphs'. In: *30th annual symposium on foundations of computer science*. IEEE. 1989, pp. 549–554 (cited on page 1).

[4] Paolo Boldi and Sebastiano Vigna. 'The webgraph framework I: compression techniques'. In: *Proceedings of the 13th international conference on World Wide Web*. 2004, pp. 595–602 (cited on pages 2, 3).

[5] Jarno N Alanko et al. 'Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes'. In: *Bioinformatics* 39.Supplement_1 (2023), pp. i260–i269 (cited on page 2).

[6] Fatemeh Almodaresi et al. 'An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search'. In: *Journal of Computational Biology* 27.4 (2020), pp. 485–499 (cited on page 3).

[7] Jarno N Alanko et al. 'Compact data structures for collections of sets'. In: *23rd Symposium on Experimental Algorithms*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik. 2025, p. 6 (cited on pages 3–5).

[8] Abraham Bookstein and Shmuel T. Klein. 'Compression of correlated bit-vectors'. In: *Information Systems* 16.4 (1991), pp. 387–400 (cited on page 3).

[9] Travis Gagie, Meng He, and Gonzalo Navarro. 'Compressed Set Representations based on Set Difference'. In: *arXiv preprint arXiv:2601.23240* (2026) (cited on pages 3–5).